

Traits

A trait is a promise: "a type with this trait has these methods". It lets you write one function that works on several types.

```
use std.io

trait Greeter {
    fn name(self) -> Text
    fn greet(self) -> Text => "hello, {self.name()}"
}

object Person {
    given: Text = "friend"
    fn name(self) -> Text => self.given
}

object Robot {
    serial: Int = 1
    fn name(self) -> Text => "unit {self.serial}"
}

impl Greeter for Person {}
impl Greeter for Robot {}

fn main() {
    let crowd = [Person { given: "Ada" }, Robot { serial: 7 }]
    for one in crowd {
        io.say(one.greet())
    }
}
```

Two things worth noticing.

`greet` has a body in the trait. That is a default: any type that takes the trait gets it free, and can replace it if it wants to.

`name` has no body, so each type must supply one. The compiler checks this and tells you exactly which method is missing.

Using a trait as a type

```
use std.io

trait Shape {
    fn area(self) -> Num
}

object Circle {
    radius: Num = 1.0
    fn area(self) -> Num => 3.14159 * self.radius * self.radius
}
```

```

object Rect {
  w: Num = 1.0
  h: Num = 1.0
  fn area(self) -> Num => self.w * self.h
}

impl Shape for Circle {}
impl Shape for Rect {}

fn total(shapes: [Shape]) -> Num {
  var sum = 0.0
  for s in shapes { sum = sum + s.area() }
  give sum
}

fn main() {
  io.say("{total([Circle { radius: 1.0 }, Rect { w: 2.0, h: 3.0 }])}")
}

```

Built in traits

A few names are understood by the compiler: `Show` for values that can be printed, `Eq` for values that compare, `Ord` for values that sort, `Hash` for values that can be map keys, and `Num` for numbers.

Exercise

Add a `perimeter` method to the `Shape` trait with a sensible default of `0.0`, implement it properly for `Rect`, and print both figures.